Writing 4: Research on Realistic 3D Buildings

Export to 3D Print extension, Sprint 1

Qiming Lyu
Team Export to 3D Print (Group 9) · CSCI 3081W, Fall 2026

November 1, 2026

Introduction

Our team's extension exports a Gopher GIS scene as matching stacks of slice images and cutting-plane images, so that the scene can be built as a physical model by 3D printing or by laser-cutting stacked layers. My Sprint 1 task was research: find out how Gopher GIS could produce realistic 3D buildings for that export, at no cost.

The technical challenge is that the prototype's buildings carry almost no 3D information. Each building is its OpenStreetMap (OSM) footprint extruded straight up to a single height with a flat roof, which GIS calls an LOD1 block model (`extrudeFootprint`). Its color is a hash of the building ID, not real data (`buildingColor`). Its height is the OSM `height` tag, else `building:levels` $\times$ 3 m, else a default of 8 m (`buildingHeight`). I counted the tags in the prototype's downloaded OSM file: 46,364 of 50,754 buildings (91.4%) fall through to the 8 m default, and none of the 109 buildings in the campus 3D view has a `height` tag. For our extension this is more than cosmetic. Every horizontal slice of a prism is the same footprint, so slicing LOD1 buildings yields a stack of identical outlines at guessed heights. Slicing only becomes meaningful once the geometry is real.

A solution has to cost nothing, cover tens of thousands of buildings, use data whose license allows export, physical fabrication and sharing, and fit the app's C++ fetch–cache–parse–mesh pipeline. My first idea, buying a 3D scanner, fails on budget and also on scale, since consumer scanners are built for objects within arm's reach. One constraint is still open: the team, instructor and TAs have not chosen a print scale. The model could show one or a few campus buildings in detail (around 1:500) or the whole 9.4 $\times$ 7.8 km data area at low detail (around 1:20,000), so my research covers both.

Approach

I first checked the problem against the code and data, which produced the counts above. The script that reproduces them, the full report, five topic notes and every source are in my research folder. I then surveyed five areas: geometry data for the whole area, capture of single buildings, appearance and textures, fabrication file formats, and C++ integration. I judged each option against primary sources (official documentation, license text, dataset metadata and live API queries) on four criteria: cost, coverage of our area, whether the license allows export and fabrication, and fit with the C++ app.

Finally, I worked out what each scale can physically show, because that decides which kinds of realism are worth building. A desktop FDM printer resolves detail at about its 0.4 mm nozzle

width [1]. Table 1 converts real sizes to model sizes.

Table 1. Real features at model scale (model mm = real m $\times$ 1000 / N).

Feature	1:500 (one building)	1:20,000 (whole area)
Whole 9.4 $\times$ 7.8 km area	—	470 $\times$ 390 mm (2 $\times$ 2 tiles on a desktop printer)
8 m building height	16 mm	0.4 mm
1.5 m window	3 mm	0.075 mm
What survives fabrication	Roof forms, setbacks, window grid, material color	Footprints, relative heights, roof color

At 1:20,000 an 8 m house is one nozzle width tall and windows vanish. Realism at that scale means correct relative heights and roof color, which is a data problem. At 1:500, roof shapes and the rhythm of a facade become visible, which is a modeling problem. Work that serves both scales should therefore come first.

Discussion

Free public lidar solves the height problem. My most important finding is that the whole data area is already covered by 2022 USGS lidar rated Quality Level 1 [2], at about 64 points per square meter across all returns [3]. It is federal public-domain data, and its 309 tiles (about 29.4 GB) download over plain HTTPS without a login [4]. For comparison, the Dutch 3DBAG pipeline, which reconstructs every building in the Netherlands, reports good results at 8–10 points per square meter [5]. That is dense enough to measure the true height of every building: compute each point's height above ground with PDAL [6], take a high percentile of the points inside each footprint, and write the result to a sidecar file keyed by OSM ID. The one wrinkle is that the vendor did not classify building points [7], so they must be selected by clipping to footprints. For campus roofs, the open-source tool roofer turns the same points plus a footprint into LOD2 buildings with real roof shapes, and it ships prebuilt binaries [8]. It is GPLv3, so it should run as a separate offline tool rather than be linked into the app.

Ready-made datasets are not good enough. Table 2 compares the alternatives to computing heights ourselves. Overture Maps has heights for 63% of the buildings in our area, but about 98% of those values are Microsoft machine-learning estimates [9], and Microsoft's own data for the area never exceeds 33.6 m [10], so it cannot represent tall buildings. The other public datasets have little height data or are out of date.

Detailed single buildings need their roofs from the air. For a detailed model of one campus building, the hard part is the roof, which no ground-level method can see. Phone LiDAR reaches only about 5 m [13]. Drones are effectively unavailable: UMN requires a flight plan approved by its UAS Safety Board at least five business days ahead [14], and parts of the East Bank lie in the Minneapolis–St. Paul airport's Class B airspace [15]. The workable design is a hybrid. The lidar roof from roofer is the base, and the walls are refined either from digitized University Archives blueprints, which UMedia holds for buildings such as Walter Library [16], or from photogrammetry. Because COLMAP's new AMD GPU support is untested on my Debian PC's RX 6600 [17],

Table 2. Building-height sources for the data area.

Source	Coverage	Basis	Verdict
OSM tags (current app)	about 9% of buildings	Mapper tags, else 8 m	The problem
Overture Maps [9]	34,485 of 54,398	Mostly Microsoft ML; maximum 33.6 m	Stopgap only
USA Structures [11]	5,633 of 41,826	NGA-sourced	Adds little
NGA 2012 buildings [12]	Whole area	2012 lidar, with roof types	Fallback; 14 years old
2022 USGS lidar, computed by us	Every footprint	2022 lidar, about 64 points/m^2	**Recommended**

free photogrammetry there means COLMAP plus OpenMVS on the CPU [18]. RealityScan is free for students [19] but needs an NVIDIA GPU on Linux [20]. Neural methods such as Gaussian splatting mostly need NVIDIA GPUs or carry research-only licenses [21, 22], and they do not yield printable meshes directly.

Licensing rules out several shortcuts, including one the app already uses. Google's photorealistic 3D tiles ban offline use and derived 3D models [23], and Cesium, Apple and Bing have similar terms. The surprise is the app's own ground imagery. Esri World Imagery falls under Esri's Master License Agreement, whose terms bar exporting its tiles for offline use or self-hosting them [24, 25], so it must not appear in any exported texture or printed layer image. Free replacements exist. The Twin Cities 2025 orthophoto is 30 cm per pixel and explicitly public domain [26]. CC0 texture libraries such as ambientCG cover walls [27]. Panoramax holds 725 recent East Bank street photos under CC BY-SA for detailed facades [28], although any texture atlas cut from them must then be released under CC BY-SA too.

No single file format serves both kinds of machine. I hoped to find one "universal" format for 3D printers and laser cutters, but none exists. Slicers read 3MF, STL and OBJ, laser software reads SVG [29], and I found no laser software that reads 3MF. Color is inconsistent even among slicers. The Prusa printers in UMN Libraries' free makerspaces [30] run PrusaSlicer, which ignores imported colors [31], so color has to be encoded as one object per color class. The universal format therefore has to be internal: one watertight solid in millimeters plus one *fabrication frame* holding the scale, vertical exaggeration, sheet thickness and laser kerf. From these the exporter writes 3MF or STL for printers and per-layer SVG cut files with matching PNG color images for laser stacks, all registered by construction (Figure 1). Terrain plus LOD1 or LOD2 buildings forms a heightfield, so a layer's outline is simply the region where the surface is above that layer's height. That is a 2D thresholding step, the same one GDAL's contour tool performs [32], with no 3D boolean operations. Only scanned meshes with overhangs need true mesh slicing, which the Apache-licensed Manifold library provides [33].

What is verified and what is not. This was research rather than a prototype, so nothing has been printed or cut yet. The tag counts are measured and reproducible, and the lidar tile list and street-photo count come from live API queries. Four things remain untested: how roofer handles points that are not classified as buildings, how long it takes on about 54,000 buildings, whether

```
OFFLINE (team scripts in Docker or conda)
  2022 lidar + footprints --PDAL----> heights.json  (LOD1, all)
  2022 lidar + footprints --roofer--> campus.city.json  (LOD2)

GOPHER GIS (C++)
  sources: Overpass, height sidecar, CityJSON, DEM
    -> SceneModel (local meters, colors)
    -> SolidBuilder + FabricationFrame (1:N, V, sheet, kerf)
    -> watertight solid in mm
         -> mesh exporters: model.3mf | model.stl | model.obj
         -> layer slicer:   layers/NNN.svg + NNN.png + manifest.json
```

Figure 1. Proposed export pipeline. Lidar processing runs offline, and the app reads its outputs through source adapters.

COLMAP's GPU path runs on my RX 6600, and the bed size and kerf of the makerspace laser cutter. If roofer stalls, there are three fallbacks: TU Delft's 3dfier, which builds lidar-based LOD1 models directly as OBJ or STL [34]; ArcGIS Pro's LOD2 extraction tool [35], which is free to UMN students [36] if their license includes the required 3D Analyst extension; and the NGA 2012 and Overture data.

Conclusion

I recommend that we do not buy a scanner or license commercial 3D data, and that we build realism from the 2022 public lidar instead. Whatever the next sprints end up looking like, two pieces of work are useful at any scale and make a good starting point. The first is to build the export core: the fabrication frame, a closed terrain slab with the current extruded buildings, STL and OBJ writers, and the heightfield slicer that writes SVG, PNG and a manifest. Export-time imagery should also switch from Esri to the public-domain Minnesota orthophoto. The second is a lidar pilot on about 1 km^2 of campus that computes true heights for the 109 campus-view buildings and runs roofer on the same area. The pilot settles roofer's unknowns cheaply before anyone commits to the whole area. Capture of a detailed building should wait until the scale is chosen. If it is wanted, Walter Library is a good candidate, because its blueprints are digitized and its masonry suits photogrammetry better than glass does.

Three decisions belong to the instructor team. The first is print scale. We propose to settle it with physical test pieces from the makerspace, a small cardboard stack and an FDM tile, rather than with arguments, as soon as the export core can produce them. The second is whether an offline preprocessing step (PDAL and roofer in Docker or conda) is acceptable outside the C++ app; it is needed partly because PDAL is no longer packaged in current Debian or Ubuntu [37]. The third affects all of Gopher GIS: Esri imagery should stay on screen only, in our extension and in any other that exports map content.

We should also confirm the makerspace laser cutter's bed size, kerf and accepted file type, and check every teammate's GPU for RealityScan. Finally, the team could add measured heights for the campus buildings to OSM by hand, with a source tag. OSM's rules allow careful manual edits of this kind [38], and the heights would then reach the Overpass feed of every future Gopher GIS team, fixing the 8 m guess at its source.

References

[1] TouchTerrain README (FDM detail limits). https://github.com/ChHarding/TouchTerrain_for_CAGEO

[2] USGS 3DEP Elevation Index, MN_CentralMissRiver_B22 quality level. https://index.nationalmap.gov/arcgis/rest/services/3DEPElevationIndex/MapServer/24

[3] OpenTopography, MN_CentralMissRiver_4_B22 dataset page. https://portal.opentopography.org/usgsDataset?dsid=MN_CentralMissRiver_4_B22

[4] USGS TNM Access API, lidar point cloud products for the data area. https://tnmaccess.nationalmap.gov/api/v1/products?bbox=-93.288,44.939,-93.168,45.009&datasets=Lidar%20Point%20Cloud%20%28LPC%29&max=200&outputFormat=JSON

[5] geoflow-bundle (3DBAG reconstruction pipeline). https://github.com/geoflow3d/geoflow-bundle

[6] PDAL height-above-ground filters (`filters.hag_nn`, `filters.hag_delaunay`). https://github.com/PDAL/PDAL/commit/9a3a9445bc3f6e452c2fadf8970df1eb75ab28bc

[7] Woolpert, MN Central Mississippi River 4 2022 Lidar Mapping Report. https://rockyweb.usgs.gov/vdelivery/Datasets/Staged/Elevation/metadata/MN_CentralMissRiver_B22/MN_CentralMissRiver_4_B22/reports/MN_Central_Miss%20_River_4_2022_Lidar_Mapping_Report.pdf

[8] 3DBAG, roofer, and roofer v1.0.0 release. https://github.com/3DBAG/roofer, https://github.com/3DBAG/roofer/releases

[9] Overture Maps, buildings guide. https://docs.overturemaps.org/guides/buildings

[10] Microsoft, GlobalMLBuildingFootprints. https://github.com/microsoft/GlobalMLBuildingFootprints

[11] USA Structures FeatureServer. https://services2.arcgis.com/FiaPA4ga0iQKduv3/ArcGIS/rest/services/USA_Structures_View/FeatureServer/0

[12] USGS, NGA US Cities: Minneapolis 2012 vector data. https://rockyweb.usgs.gov/vdelivery/Datasets/Staged/Elevation/Non_Standard_Contributed/NGA_US_Cities/Minneapolis_MN/Minneapolis_20120420-30/VEC/

[13] iPad Pro LiDAR accuracy study, *Sensors*. https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9657006/

[14] University of Minnesota, Drones policy. https://policy.umn.edu/operations/drones

[15] FAA, Class Airspace layer. https://services6.arcgis.com/ssFJjBXIUyZDrSYZ/arcgis/rest/services/Class_Airspace/FeatureServer/0

[16] UMedia, University Archives blueprint record. https://lib-umedia-prd-app-01.oit.umn.edu/item/p16022coll244:686.json

[17] COLMAP 4.2.0 release notes. https://github.com/colmap/colmap/releases/tag/4.2.0

[18] OpenMVS. https://github.com/cdcseacave/openMVS

[19] RealityScan license. https://www.realityscan.com/license

[20] Epic Games, RealityScan for Linux. https://dev.epicgames.com/documentation/en-us/realityscan/realityscan-for-linux

[21] nerfstudio. https://github.com/nerfstudio-project/nerfstudio

[22] Inria, Gaussian-Splatting license. https://github.com/graphdeco-inria/gaussian-splatting/blob/main/LICENSE.md

[23] Google, Map Tiles API policies. https://developers.google.com/maps/documentation/tile/policies

[24] Esri, World Imagery item description. https://www.arcgis.com/sharing/rest/content/items/10df2279f9684e4a9f6a7f08febac2a9?f=json

[25] Esri, Terms of use summary. https://www.esri.com/content/dam/arcgisonline/docs/tou_summary.pdf

[26] Minnesota Geospatial Commons, Twin Cities 2025 orthoimagery metadata. https://mnatlas.org/metadata/2025_7_county.html

[27] ambientCG, License. https://docs.ambientcg.com/license/

[28] Panoramax search API, East Bank query. https://api.panoramax.xyz/api/search?bbox=-93.245,44.968,-93.222,44.982&limit=1000

[29] VisiCut manual. https://github.com/t-oster/VisiCut/wiki/VisiCut-manual

[30] UMN Libraries, Makerspaces guide. https://libguides.umn.edu/makerspaces/home

[31] Prusa forum, “Import model with color.” https://forum.prusa3d.com/forum/prusaslicer/import-model-with-color/

[32] GDAL, `gdal_contour`. https://gdal.org/en/stable/programs/gdal_contour.html

[33] Manifold. https://github.com/elalish/manifold

[34] TU Delft, 3dfier. https://github.com/tudelft3d/3dfier

[35] Esri, Extract LOD2 Buildings tool reference. https://pro.arcgis.com/en/pro-app/latest/tool-reference/3d-analyst/extract-lod2-buildings.htm

[36] U-Spatial, Software (ArcGIS Pro for UMN). https://rc.umn.edu/uspatial/software

[37] Repology, PDAL versions. https://repology.org/project/pdal/versions

[38] OpenStreetMap Wiki, Import guidelines. https://wiki.openstreetmap.org/wiki/Import/Guidelines